

Advanced Machine Learning

DATA 642

Lecture 6 — Special Considerations

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

PCA and Singular Value Decomposition (SVD)

- **Eigen-decomposition Approach:**

- Given the covariance matrix $\mathbf{S} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top$, compute the eigenvalues λ_i and corresponding eigenvectors $\mathbf{u}_i$.
- The eigenvectors $\mathbf{u}_i$ represent the principal components, and the eigenvalues λ_i represent the variance along each principal component.

- **Singular Value Decomposition (SVD) Approach:**

- Compute the SVD of the centered data matrix $\mathbf{X}$: $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.
- The right singular vectors $\mathbf{V}$ represent the principal components, and the singular values along the diagonal of $\mathbf{\Sigma}$ represent the square roots of the eigenvalues of $\mathbf{X}^\top \mathbf{X}$.

Both approaches provide equivalent results, offering a direct connection between the maximum variance perspective of PCA and the SVD of the data matrix $\mathbf{X}$.

PCA and Low-Rank Approximations

Having established the bridge between PCA and SVD, another interpretation of the PCA method becomes readily available. Because $\hat{\mathbf{X}}$ is the best m rank matrix approximation of $\mathbf{X}$ in the Frobenius sense, the quantity

$$||\hat{\mathbf{X}} - \mathbf{X}||_F^2 = \sum_{n=1}^N ||\hat{\mathbf{x}}_n - \mathbf{x}_n||^2 \quad (1)$$

is minimum.

This implies that the sum of squared errors between the original data points $\mathbf{x}_n$ and their approximations $\hat{\mathbf{x}}_n$ is minimized under PCA, ensuring that the principal components capture as much information as possible while reducing the dimensionality of the data.

Latent Variable Perspective

So far we derived PCA without any notion of a probabilistic model using the maximum-variance and the projection perspectives. A probabilistic model would offer us more flexibility and useful insights. More specifically, a probabilistic model would

- Come with a likelihood function, and we can explicitly deal with noisy observations (which we did not even discuss earlier)
- Allow us to do Bayesian model comparison via the marginal likelihood
- View PCA as a generative model, which allows us to simulate new data
- Allow us to make straightforward connections to related algorithms
- Deal with data dimensions that are missing at random by applying Bayes' theorem
- Give us a notion of the novelty of a new data point
- Give us a principled way to extend the model, e.g., to a mixture of PCA models
- Have the PCA we derived in earlier sections as a special case
- Allow for a fully Bayesian treatment by marginalizing out the model parameters

The history of Probabilistic PCA

By introducing a continuous-valued latent variable $\mathbf{z} \in \mathbb{R}^M$ it is possible to phrase PCA as a probabilistic latent-variable model. Tipping and Bishop **probabilistic PCA** (1999) proposed this latent-variable model as probabilistic PCA (PPCA). PPCA addresses most of the aforementioned issues, and the PCA solution that we obtained by maximizing the variance in the projected space or by minimizing the reconstruction error is obtained as the special case of maximum likelihood estimation in a noise-free setting.

The model of Probabilistic PCA

In PPCA, we explicitly write down the probabilistic model for linear dimensionality reduction. For this we assume a continuous latent variable $\mathbf{z} \in \mathbb{R}^M$ with a standard-normal prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and a linear relationship between the latent variables and the observed $\mathbf{x}$ data where

$$\mathbf{x} = \mathbf{B}\mathbf{z} + \boldsymbol{\mu} + \epsilon \in \mathbb{R}^D, \quad (2)$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ is Gaussian observation noise and $\mathbf{B} \in \mathbb{R}^{D \times M}$ and $\boldsymbol{\mu} \in \mathbb{R}^D$ describe the linear/affine mapping from latent to observed variables. Therefore, PPCA links latent and observed variables via

$$p(\mathbf{x}|\mathbf{z}; \mathbf{B}, \boldsymbol{\mu}, \sigma^2) = \mathcal{N}(\mathbf{x}|\mathbf{B}\mathbf{z} + \boldsymbol{\mu}, \sigma^2 \mathbf{I}) \quad (3)$$

Overall, PPCA induces the following generative process:

$$\mathbf{z}_n \sim N(\mathbf{z}|\mathbf{0}, \mathbf{I}) \tag{4}$$

and

$$\mathbf{x}_n \mid \mathbf{z}_n \sim N(\mathbf{x}|\mathbf{B}\mathbf{z}_n + \boldsymbol{\mu}, \sigma^2\mathbf{I}). \tag{5}$$

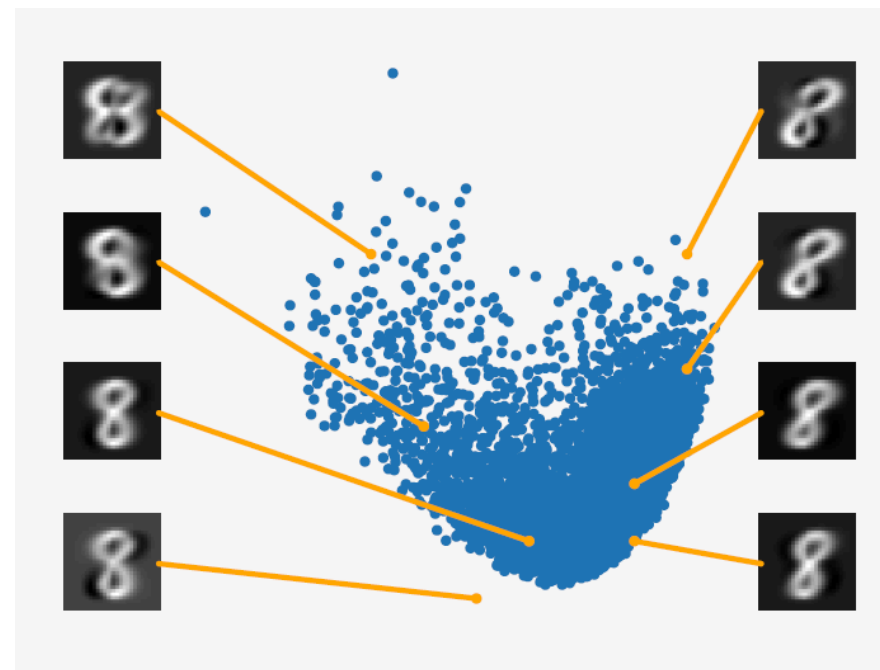
To generate a data point that is typical given the model parameters, we perform **ancestral sampling**. We first sample a latent variable $\mathbf{z}_n$ from $p(\mathbf{z})$. Then we use $\mathbf{z}_n$ in Equation $p(\mathbf{x}|\mathbf{z}; \mathbf{B}, \boldsymbol{\mu}, \sigma^2) = \mathcal{N}(\mathbf{x}|\mathbf{B}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$ to sample a data point conditioned on the sampled $\mathbf{z}_n$, i.e., $\mathbf{x}_n \sim p(\mathbf{x}|\mathbf{z}_n; \mathbf{B}, \boldsymbol{\mu}, \sigma^2)$.

This generative process allows us to write down the probabilistic model (i.e., the joint distribution of all random variables) as

$$p(\mathbf{x}, \mathbf{z}|\mathbf{B}, \boldsymbol{\mu}, \sigma^2) = p(\mathbf{x}|\mathbf{z}, \mathbf{B}, \boldsymbol{\mu}, \sigma^2)p(\mathbf{z}). \quad (6)$$

An illustration of PPCA

- The following figure shows the latent coordinates of the MNIST digits "8" found by PCA when using a two-dimensional principal subspace (blue dots).
- We can query any vector $\mathbf{z}_*$ in this latent space and generate an image $\tilde{\mathbf{x}}_* = \mathbf{B}\mathbf{z}_*$ that resembles the digit "8". This is under the assumption that $\boldsymbol{\mu} = \mathbf{0}$ and the covariance is the identity matrix.



- We show eight of such generated images with their corresponding latent space representation. Depending on where we query the latent space, the generated images look different (shape, rotation, size, etc.).